Algorithmique I

Corrigé du bilan 2025

Épreuve du 6 novembre 2025 · Questions 1 à 4, sous-questions incluses

Les algorithmes des énoncés anciens ont été transcrits vers le LDA du syllabus actuel. Les hypothèses non précisées par les énoncés sont signalées explicitement.

Question 1 : Simplification et clarification (5 points)

[1, p. 1].. Deux algorithmes indépendants.

a. `chaleur`. Le booléen `chaud` et ses comparaisons à `true`/`false` sont inutiles.

```
void chaleur()
    float temperature
    read temperature
    if temperature > 25
        print "Il fait chaud"
    else
        print "Il ne fait pas chaud"
```

Contrôles : 26 → « Il fait chaud » ; 25 → « Il ne fait pas chaud ». La frontière est strictement supérieure à 25.

b. `deliberation`. Dans le programme initial, la condition `note` $\leq$ `8` ne peut jamais être atteinte : elle arrive après `note < 10`. On la teste d'abord pour respecter le sens suggéré par les trois messages.

```
void deliberation()
    int note
    read note
    if note ≤ 8
        print "échec"
    else if note < 10
        print "étudiant à délibérer"
    else
        print "réussite"
```

Contrôles : 8 → « échec » ; 9 → « étudiant à délibérer » ; 10 → « réussite ». **Hypothèse :** les seuils voulus sont 0-8, 9 et 10-20. L'énoncé ne comporte pas de correction officielle justifiant ce découpage ; c'est l'interprétation cohérente avec les trois libellés.

Question 2 : Traçage d'algorithme (4 points)

[1, p. 2].. L'algorithme `naissance` lit `mois=11`, `année=1990`.

Trace de `naissance` (lignes 1 à 21). Les variables sont déclarées aux lignes 2 à 4. Les instructions `print` sont relevées à part.

Ligne	mois	année	n	bissextile	ordinal
2-4 - Déclarations	-		- -	-	-
5 - `read mois, année`	11	1990	-	-	-
6 - `n = (mois-1) DIV 3`	11	1990	3	-	-
7 - `bissextile = estBissextile(année)`	11	1990	3	`false`	-
13 - `ordinal = "quatrième"`	11	1990	3	`false`	`"quatrième"`

À la ligne 9, le `switch` choisit `default` puisque `n=3` ; la ligne 13 est la seule affectation exécutée dans ce bloc. À la ligne 16, la condition `bissextile` est fausse ; la ligne 19 s'exécute.

Trace de `estBissextile(1990)` (lignes 23 à 25).

Ligne du module	a
23 - Paramètre `a`	1990

Retour (ligne 24) : `false`, car `1990 MOD 4 = 2` et `1990 MOD 400 = 390`. Aucun `print` dans ce module.

Texte affiché par `naissance` (trois instructions distinctes) :

Ligne	Texte affiché
8	`Vous êtes né(e) au`
15	`quatrième trimestre d'une année`
19	`non bissextile`

```
Vous êtes né(e) au
quatrième trimestre d'une année
non bissextile
```

Question 3 : Petits problèmes de matricule (3 points)

[1, p. 3].. On suppose le matricule un entier de cinq chiffres, comme demandé.

a. Comparer le premier et le dernier chiffre. Le premier s'obtient par `DIV 10000`, le dernier par `MOD 10` :

```
void matricule1()
    int matricule
    read matricule
    if matricule DIV 10000 == matricule MOD 10
        print "Le premier chiffre est égal au dernier "
    else
        print "Le premier chiffre est différent du dernier "
```

Contrôles de l'énoncé : 65546 → égaux ; 67781 → différents. Les espaces de fin des deux chaînes proviennent de l'énoncé.

b. Calculer le résultat à partir d'un matricule personnel. Aucune valeur individuelle n'étant fournie, le résultat ne peut pas être chiffré. Voici la **méthode exacte**.

```
// Pour un matricule M à cinq chiffres :
debut = M DIV 1000
fin = M MOD 100
```

```
if debut > fin
    resultat = debut + 100*fin
else
    resultat = fin + 100*debut

print resultat
```

En posant `D=M DIV 1000` (les deux premiers chiffres) et `F=M MOD 100` (les deux derniers), la valeur affichée est `D + 100*F` lorsque `D>F`, et `F + 100*D` sinon. Introduire **son propre matricule** dans cette expression pour obtenir un résultat numérique ; aucune valeur personnelle n'est inventée ici.

Question 4 : Le carré (8 points)

[1, p. 4].. Pour une diagonale `d` en cm, le côté vaut `c = sqrt(2)*d/2` et l'aire vaut `c*c = d*d/2` en cm².

a. `coteCarre` : une valeur reçue en paramètre, une valeur retournée.

```
float coteCarre(float diagonale)
    float cote
    cote = sqrt(2)*diagonale/2
    return cote
```

b. `aireCarre` : lecture, test, appel obligatoire à `coteCarre`, puis affichage.

```
void aireCarre()
    float diagonale, cote, aire
    read diagonale
    if diagonale ≥ 0
        cote = coteCarre(diagonale)
        aire = cote*cote
        print aire, " cm²"
    else
        print "Impossible de calculer l'aire du carré"
```

Contrôles : `d=0` → `0 cm²` ; `d=2` → côté `sqrt(2)` cm et aire `2 cm²` ; `d=-1` → message d'impossibilité, sans appel au module. Les valeurs dans `coteCarre` sont calculées, puis **retournées**, non affichées. Les affichages de la sous- question b ne sont pas imposés mot pour mot, contrairement aux chaînes des questions précédentes.

Références

[1] ESI, HE2B, *Algorithmique I : bilan 2025*, Bruxelles, Belgique, 2025.